

Autonomous Data Quality Investigation Agent - Agentic AI - Gemini 1.5 Flash

Project Description:

The Autonomous Data Quality Investigation Agent harnesses agentic AI to automatically diagnose, investigate, recommend fixes for, and validate data quality defects in tabular datasets. Unlike passive data quality tools that only trigger alerts, the platform includes a Profiling Agent, which discovers nulls, cardinality issues, and duplicate transaction IDs, an Anomaly Detection Agent, which flags statistical outliers using Z-scores and Interquartile Ranges, a Schema Analysis Agent, which detects type mismatches and non-nullable field breaches, a Root Cause Agent, which correlates multi-agent evidence into probabilistic hypotheses, a Correction Recommendation Agent, which produces a prioritized, risk-rated action plan pending human approval, and a Validation Agent, which independently verifies post-correction datasets against baseline metrics.

Utilizing LangGraph to orchestrate the multi-agent workflow and Google's Gemini API for human-readable interpretation of deterministic findings, the platform processes uploaded CSV datasets to deliver a full investigation report in seconds. Built with a FastAPI backend and a dark-themed HTML/CSS/JavaScript dashboard, and enforcing a human-in-the-loop approval gate before any correction is validated, the platform empowers data engineers and analysts to resolve data quality defects with confidence and full auditability.

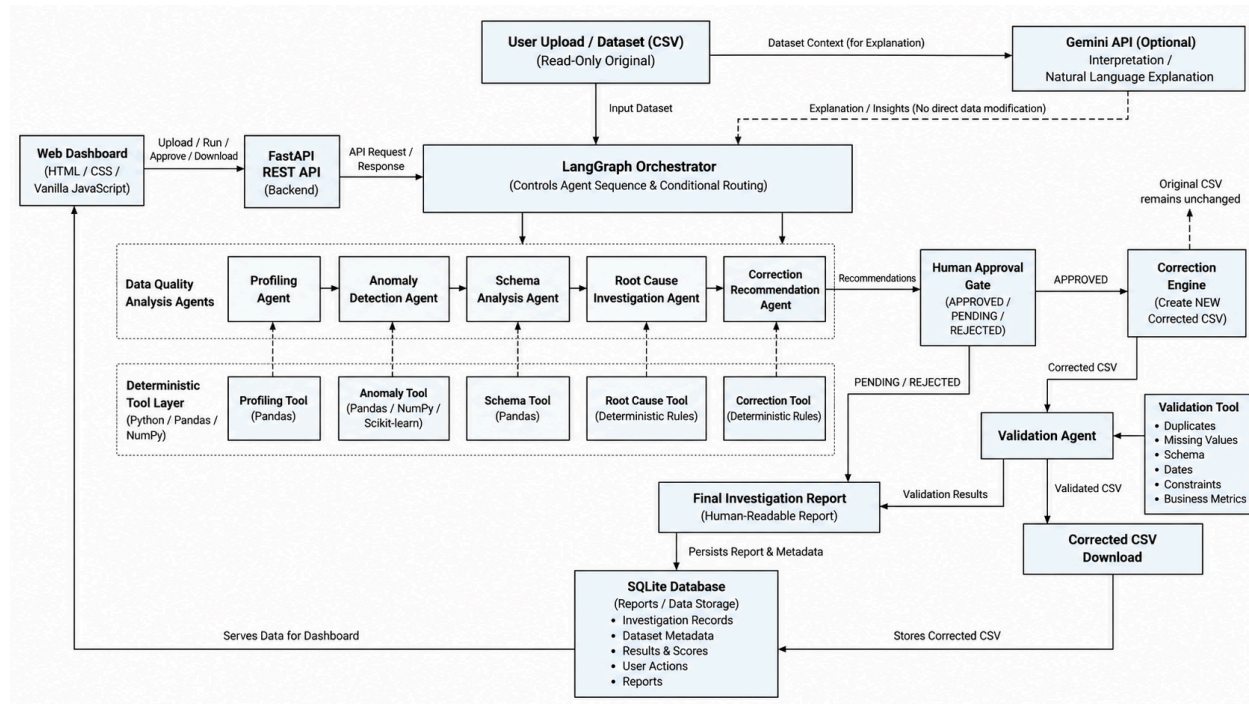
Scenarios

Scenario 1: A user uploads a problematic sales CSV with duplicate transaction IDs and missing values. The Profiling Agent surfaces the null and duplicate counts, the Anomaly Agent flags outlier revenue values using IQR, and the Root Cause Agent correlates both findings into a single hypothesis — all before any correction is recommended.

Scenario 2: An analyst reviews the Correction Recommendation Agent's prioritized action plan on the dashboard and clicks "Approve & Validate." The workflow transitions the approval status to APPROVED, runs the Validation Agent against the corrected dataset, and returns a PASS/PARTIAL/FAILED verdict with before/after metric comparisons for duplicates, nullability, and revenue recalculations.

Scenario 3: An analyst is not yet ready to apply a correction and clicks "Reject" instead. The workflow halts at the Human Approval Gate, skips the Validation Agent entirely, and the Final Workflow Report records the investigation findings with a PENDING or REJECTED status for later re-review.

Technical Architecture:



Description: The Autonomous Data Quality Investigation Agent uses a modular multi-agent architecture orchestrated by LangGraph to deliver auditable, evidence-based data quality diagnosis. The frontend provides a dark-themed dashboard built with HTML, CSS, and vanilla JavaScript, communicating with a FastAPI backend over REST. Each agent (Profiling, Anomaly, Schema, Root Cause, Correction, Validation) first runs a deterministic Pandas/NumPy tool to compute facts and numbers, then optionally calls the Gemini API purely to explain those facts in natural language — the LLM never invents statistics, root causes, or corrective actions. A Human Approval Gate sits between the Correction Agent and the Validation Agent, ensuring no correction is validated without explicit human sign-off.

Pre-requisites:

- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **LangGraph Orchestration Familiarity:** [LangGraph Documentation](#)
- **Google Gemini API Familiarity:** [Gemini API Documentation](#)
- **Pandas & NumPy Proficiency:** [Pandas Documentation](#), [NumPy Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [FastAPI Installation Guide](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Gemini for interpreting deterministic data quality findings.
- **Activity 1.3:** Define the multi-agent architecture, detailing interactions between the frontend, FastAPI backend, LangGraph orchestration layer, and each diagnostic agent.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for FastAPI, LangGraph, and the Gemini API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core diagnostic agents: Profiling, Anomaly Detection, Schema Analysis, Root Cause Investigation, Correction Recommendation, and Validation.
- **Activity 2.2:** Implement the FastAPI backend to manage routing, dataset upload handling, and workflow execution, ensuring smooth API interactions.

Milestone 3: Main Application Development

- **Activity 3.1:** Write the main application logic in main.py and app/graph/workflow.py, establishing the LangGraph state graph, routing rules, and API endpoints, and integrating agent responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive dashboard layout.
- **Activity 4.2:** Create dynamic rendering logic to display the workflow timeline, approval gate, and validation results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Run the FastAPI application locally via Uvicorn, exposing the dashboard and Swagger API docs over a local URL.

Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Gemini for our data quality interpretation needs. This involves researching the capabilities and performance of available models, ensuring that the chosen model aligns well with the application's objective of explaining deterministic findings without inventing statistics or root causes.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Gemini LLM, you need a valid API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1** — Create a Google AI Studio Account: Visit <https://aistudio.google.com> and sign up for a free account using your Google account.
- **Step 2** — Navigate to API Keys: After logging in, open the API Keys section and generate a new key for the google-genai SDK.
- **Step 3** — Add the Key to Your Project: Create a .env file in the project root (based on .env.example) and paste the key as shown below:

```
LLM_API_KEY=your_gemini_api_key_here
```

```
LLM_MODEL=gemini-2.0-flash
```

- **Step 4** — Verify the Key is Loaded: The app.utils.config module loads these values automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
LLM_API_KEY = os.getenv("LLM_API_KEY", "")
```

- **Important** — Never Commit Your API Key: Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository.

```
autonomous-data-quality-agent > app > utils > llm_client.py > ...
22
23 from __future__ import annotations
24
25 import logging
26 from typing import Optional
27
28 from app.utils.config import settings
29
30 logger = logging.getLogger(__name__)
31
32 # -----
33 # Try importing the Gemini SDK; set a flag if unavailable
34 # -----
35 try:
36     from google import genai as _genai          # type: ignore
37     from google.genai import types as _gtypes   # type: ignore
38     _GEMINI_AVAILABLE = True
39 except ImportError:
40     _genai = None    # type: ignore
41     _gtypes = None   # type: ignore
42     _GEMINI_AVAILABLE = False
43
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to be researched to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the investigation platform — interpreting profiling, anomaly, schema, root-cause, correction, and validation results in plain language.
- **Explore Gemini's Model Documentation:** Examine the available Gemini models via the google-genai SDK, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on interpretation quality, response speed, and reliability for structured explanatory text.
- **Select the Optimal Model:** Configure the model through the LLM_MODEL environment variable, with the client wrapper (app/utils/llm_client.py) designed to fall back to deterministic rule-based interpretation whenever the SDK, key, or API call is unavailable.

```

omous-data-quality-agent > app > utils > llm_client.py > ...
class LLMClient:
    _DEFAULT_MODEL = "gemini-1.5-flash"

    def __init__(self) -> None:
        self._configured = False
        self._model_name = ""
        self._client = None # google.genai.Client

        if not _GEMINI_AVAILABLE:
            logger.info(
                "[LLMClient] google-genai SDK not installed. "
                "LLM interpretation will be skipped."
            )
            return

        api_key = settings.LLM_API_KEY
        if not api_key:
            logger.info(
                "[LLMClient] LLM_API_KEY not set. "
                "LLM interpretation will be skipped."
            )
            return

        try:
            self._client = _genai.Client(api_key=api_key) # type: ignore[union-attr]
            self._model_name = settings.LLM_MODEL or self._DEFAULT_MODEL
            self._configured = True
            logger.info(
                "[LLMClient] Gemini client configured with model '%s'.",
                self._model_name,
            )
        except Exception as exc:
            logger.warning("[LLMClient] Gemini configuration failed: %s", exc)

```

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Create a visual representation of the application architecture, including the frontend (HTML, CSS, JavaScript), FastAPI backend, LangGraph state graph, and the six diagnostic agents.
- Detail Frontend Functionality: Outline how users interact with the application — uploading a CSV dataset, running the investigation, reviewing the workflow timeline, and approving or rejecting the correction recommendation.
- Outline Backend Responsibilities: Specify how the FastAPI backend handles incoming POST requests to /investigate and /upload, validates uploaded files, and passes the dataset path into the LangGraph workflow.
- Describe AI Integration Points: Define how each agent runs its deterministic Pandas/NumPy tool first, then optionally calls the Gemini client to interpret the findings in natural language, with the LLM never permitted to override the deterministic numbers or verdicts.

Activity 1.4: Set Up the Development Environment

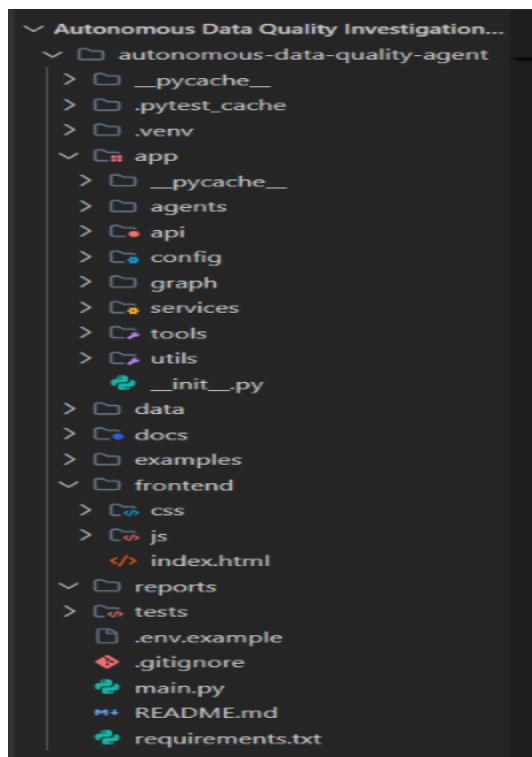
- Install Python and Pip: Ensure Python 3.10+ is installed on your machine along with pip for dependency management.
- Create a Virtual Environment: Set up a virtual environment using venv to isolate project dependencies.
- Install FastAPI and Uvicorn: Use pip to install the web framework and ASGI server:

- **Install LangGraph:** Install the orchestration library used to build the agent workflow:

- Install the Gemini SDK: Install the google-genai package for LLM interpretation:

- Install python-dotenv: Install dotenv support for secure API key management:

- **Set Up Application Structure:** Create the project directory structure including folders for `app/agents`, `app/api`, `app/graph`, `app/tools`, `app/utils`, `frontend`, `data`, `tests`, and main application files (`main.py`, `requirements.txt`).



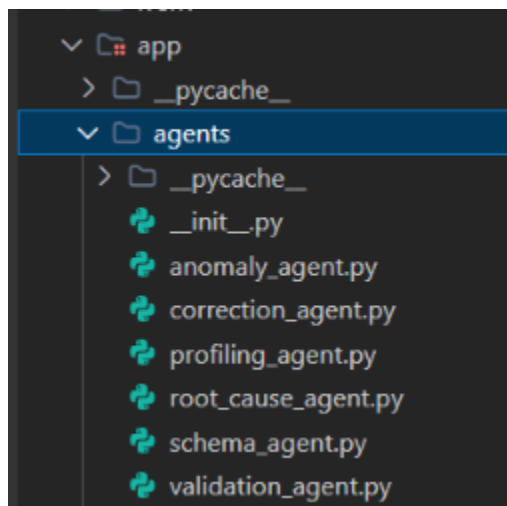
Milestone 2: Core Functionalities Development

This milestone focuses on developing the core diagnostic agents responsible for identifying, analyzing, and validating data quality issues. The agents work together to detect anomalies, schema violations, root causes, and recommend appropriate corrections. A FastAPI backend is implemented to handle dataset uploads, API routing, and execution of the LangGraph workflow. This establishes the main functional foundation of the Autonomous Data Quality Investigation Agent.

Activity 2.1: Develop Core Diagnostic Agent Features

Implement the six diagnostic agents, each pairing a deterministic tool with an optional LLM-based interpretation layer.

- **Profiling Agent:** Runs the deterministic Pandas profiling tool to discover nulls, cardinality, and duplicate transaction IDs, then interprets the findings via Gemini or a rule-based fallback.
- **Anomaly Detection Agent:** Runs Z-score and Interquartile Range calculations in NumPy/Pandas to flag statistical outliers, then explains them without inventing new statistics.
- **Schema Analysis Agent:** Compares the dataset against an expected schema (app/config/schemas/sales_schema.json) to detect type mismatches, non-nullable field breaches, and date format errors.
- **Root Cause Agent:** Correlates evidence from the Profiling, Anomaly, and Schema agents into probabilistic root-cause hypotheses with confidence ratings.
- **Correction Recommendation Agent:** Generates a prioritized, risk-rated action plan, explicitly flagging every data-altering step with `human_approval_required: True`.
- **Validation Agent:** Independently re-checks a post-correction dataset against baseline metrics, producing an authoritative PASS/PARTIAL/FAILED/NOT_APPLICABLE verdict that the LLM cannot override.



Autonomous Data Quality Investigation
 Multi-Agent Root Cause Analysis & Validation Engine

API ONLINE

DETERMINED

Multi-Agent Diagnostic Evidence

EXPANDABLE DETAILS


Profiling Agent Findings

Dataset 'problematic_sales_demo_20260923.111218' contains 51 rows and 10 columns. Overall completeness is 99.6% (2 missing cells out of 510 total). There are 1 fully duplicate rows (1.96% of all rows). Additionally, 2 duplicate transaction_id values were found.


Anomaly Detection Evidence

Anomaly detection on 'problematic_sales_demo_20260923.111218' (51 rows) identified 8 anomalous finding(s): 4 HIGH, 3 MEDIUM, 1 LOW severity. The most significant numerical outlier is column 'total_amount' with 5 IQR outliers (9.80%). 3 time-series metric period(s) showed HIGH-severity deviation from the daily baseline.


Schema Analysis Violations

SUMMARY: The dataset 'problematic_sales_demo_20260923.111218.csv' contains schema violations causing schema validation to fail. A total of 8 schema issues were detected (0 Critical, 5 High, 3 Medium, 0 Low) across 51 rows. KEY FINDINGS: • [HIGH] Type Mismatch: Column 'quantity' — Column 'quantity' expects logical type 'integer', but actual dtype is 'object'. • [HIGH/MEDIUM] Format Issue: Column 'transaction_date' — 1 unparseable dates, 1 inconsistent formats. • [HIGH] Nullability Violation: Column 'customer_id' contains 1 null/missing values in a non-nullable field. • [HIGH] Nullability Violation: Column 'quantity' contains 1 null/missing values in a non-nullable field. • [HIGH/MEDIUM] Constraint Violation: Column 'unit_price' — Column 'unit_price' has 1 values violating constraint (> 0.0). • [HIGH/MEDIUM] Constraint Violation: Column 'region' — Column 'region' contains 1 values outside allowed domain. • [HIGH/MEDIUM] Constraint Violation: Column 'payment_method' — Column 'payment_method' contains 1 values outside allowed domain. • [HIGH/MEDIUM] Constraint Violation: Column 'sales_channel' — Column 'sales_channel' contains 1 values outside allowed domain. SCHEMA CONCERNS: • Severe schema structural failures (0 critical, 5 high) require resolution prior to downstream processing.


Root Cause Bayesian Analysis

Categorical columns contain values outside approved business domain reference lists.

Activity 2.2: Implement the FastAPI Backend

Define Routes in FastAPI:

- Set up routes in app/api/routes.py for health checks, dataset upload, workflow investigation, and corrected-dataset download.

Process User Input:

- Accept multipart/form-data CSV uploads via POST /upload, validating file extension, size (50 MB limit), and content before saving with a sanitized, timestamped filename.
- Accept JSON investigation requests via POST /investigate, containing the dataset path, schema name, approval status, and optional corrected dataset path.

Integrate Workflow Calls:

- Within the /investigate route handler, call run_data_quality_workflow() to execute the full LangGraph state graph across all six agents and the human approval gate.
- Return the compiled final_report from the workflow state as the API response, raising a 500 error if the report could not be compiled.

Autonomous Data Quality Investigation Agent 1.1.0 OAS 2.1

[OpenAPI Spec](#)

An agentic AI system that detects data-quality problems, investigates root causes, recommends corrective actions, validates results, and generates investigation reports via LangGraph orchestration.

System

GET	/health	Service Health Check	
-----	---------	----------------------	---

Dataset

POST	/upload	Upload Local CSV Dataset	
------	---------	--------------------------	---

GET	/download/corrected	Download Corrected CSV Dataset	
-----	---------------------	--------------------------------	---

Investigation

POST	/investigate	Execute Data Quality Investigation Workflow	
------	--------------	---	---

Schemas

Body_upload_dataset_upload_post > Expand all object

HTTPValidationError > Expand all object

HealthResponse > Expand all object

InvestigationRequest > Expand all object

InvestigationResponse > Expand all object

UploadResponse > Expand all object

ValidationError > Expand all object

Milestone 3: Main Application Development

Milestone 3 focuses on creating and refining the core logic of main.py and the LangGraph workflow, which serve as the backbone of the Autonomous Data Quality Investigation Agent. In this milestone, the team sets up each agent's execution order through the state graph, establishes reliable routing and API interaction, and conducts testing to ensure each phase performs as expected.

Activity 3.1: Writing the Main Application Logic

Define the Core Routes in main.py:

- Establish the FastAPI application factory with a lifespan hook that logs startup status, including whether the LLM is configured:
- Mount the frontend as static files and serve the dashboard at the root route:

Define the LangGraph State Graph in app/graph/workflow.py:

- Build the state graph in the order Profiling → Anomaly → Schema → Root Cause → Correction → Human Approval Gate → Validation → Final Report, using conditional routing functions (such as route_after_profiling) to determine whether downstream nodes run.
- Skip the Correction and Validation nodes entirely for a clean dataset with no detected defects, and skip Validation specifically whenever the approval status is not APPROVED or no corrected dataset path is supplied.

Set Up Route Handlers for Each Feature:

- For the /upload route, implement path-traversal protection, extension validation, and a 50 MB size limit before writing the file to data/uploads/.
- For the /investigate route, implement error handling that returns a 500 response with a descriptive detail message if the workflow raises an exception.
- For the /download/corrected route, implement strict path validation confined to the project's data/ directory before serving the file.

```
autonomous-data-quality-agent > main.py > ...
61
62 # Register API routers
63 app.include_router(router)
64
65 # -----
66 # Frontend Static Files & Dashboard Route
67 # -----
68
69 if FRONTEND_DIR.exists():
70     app.mount("/static", StaticFiles(directory=FRONTEND_DIR), name="static")
71
72 @app.get("/", tags=["Frontend"], include_in_schema=False)
73 async def serve_frontend():
74     """Serve the Data Quality Investigation Web Dashboard."""
75     index_file = FRONTEND_DIR / "index.html"
76     if index_file.exists():
77         return FileResponse(index_file)
78     return {"message": "Frontend index.html not found"}
79
80
81 # -----
82 # Direct execution
83 # -----
84
85 if __name__ == "__main__":
86     uvicorn.run(
87         "main:app",
88         host=settings.APP_HOST,
89         port=settings.APP_PORT,
90         reload=settings.APP_DEBUG,
91         log_level=settings.LOG_LEVEL.lower(),
92     )
93
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly, enterprise-style dashboard for the Autonomous Data Quality Investigation Agent. This involves designing a dark, glassmorphic layout with responsive HTML and CSS, and creating dynamic content rendering with vanilla JavaScript to display agent execution states, approval actions, and validation results.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

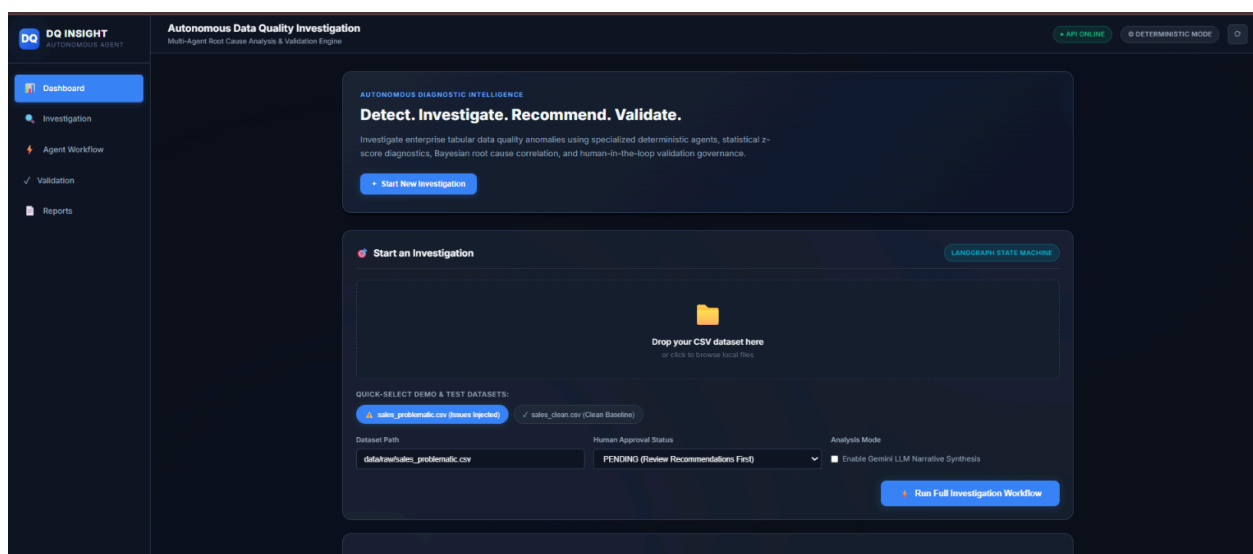
- Develop the main HTML file (frontend/index.html) as the single-page dashboard, including a hero section, an investigation input card, a workflow timeline card, an approval gate banner, a correction result section, an audit section, a validation section, and a reports section.
- Use semantic HTML elements to keep the structure organized and ensure the dashboard is accessible for all users.

Design a Responsive Layout Using CSS:

- Create CSS stylesheets (frontend/css/styles.css, components.css, responsive.css) implementing a dark, high-contrast, glassmorphic aesthetic.
- Use flexbox and grid layouts to organize the workflow timeline, defect distribution charts, and before/after comparison tables.
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

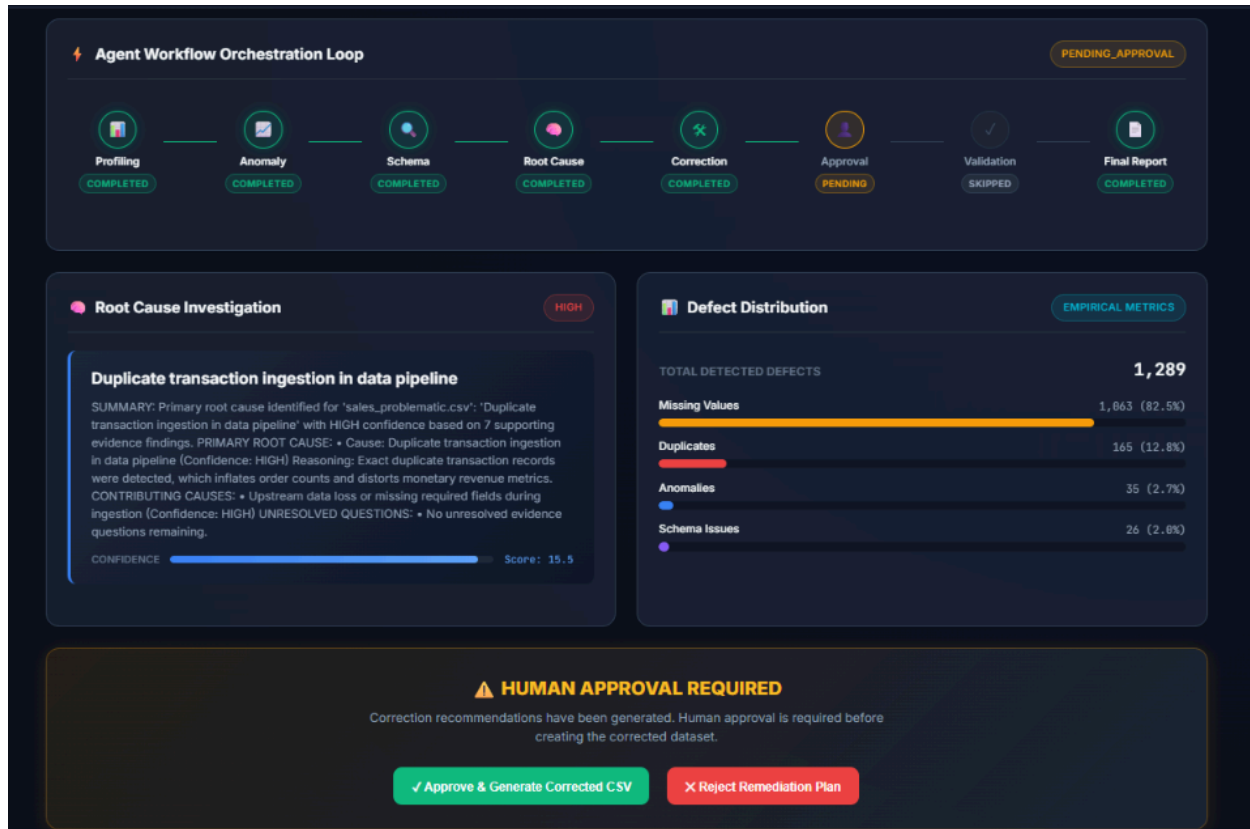
- Workflow Timeline: a dynamic stepper visualizing each agent's execution state (COMPLETED, PENDING, RUNNING, SKIPPED, FAILED).
- Approval Gate Banner: an interactive human-in-the-loop control offering one-click "Approve & Validate" or "Reject" actions.
- Defect Distribution & Validation Charts: pure SVG/Canvas data visualizations with zero external framework dependencies.
- Before/After Comparison Table: a side-by-side metric table tracking duplicates, nullability, type mismatches, and revenue recalculations.



Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Dataset Submission Asynchronously:

- Attach a submit handler (frontend/js/app.js, api.js) that sends the dataset path and options as a JSON POST request to /investigate via the Fetch API.
- During processing, update the workflow timeline in real time as each agent's state is returned in the response.

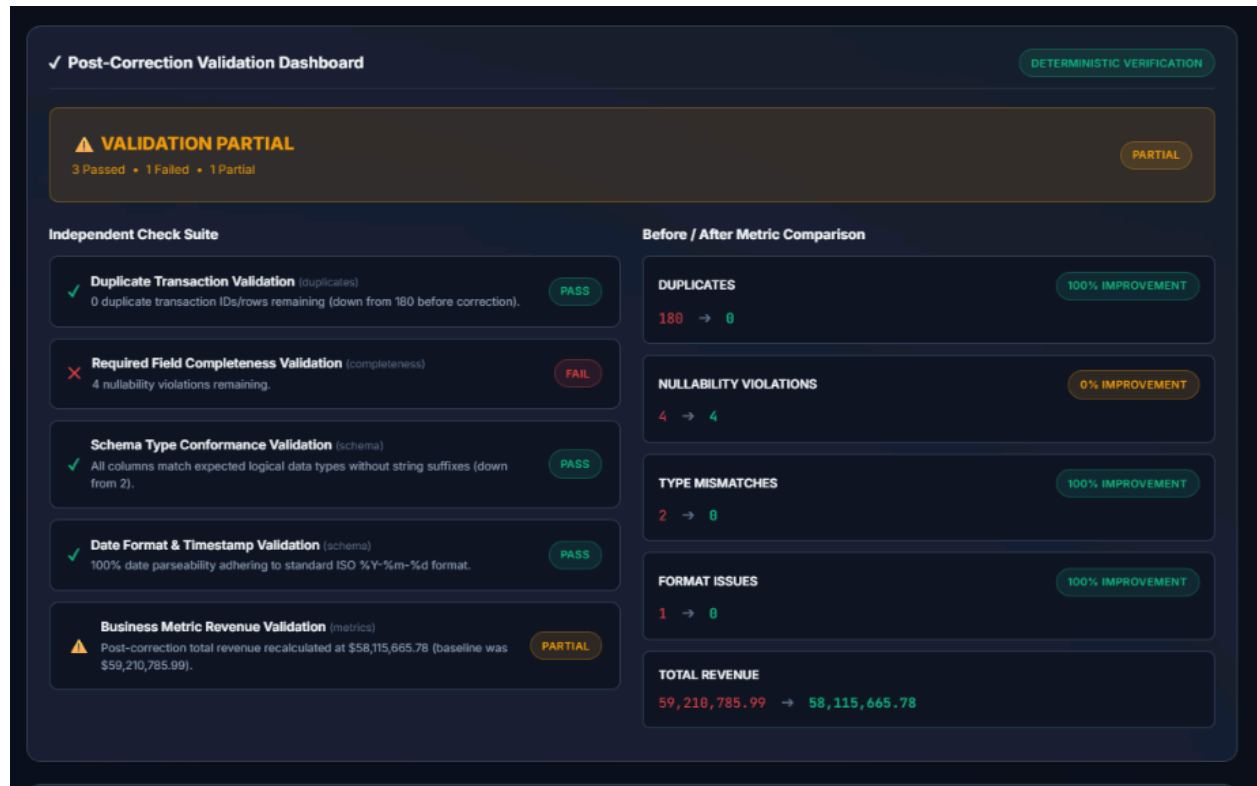


Render Results Dynamically:

- Implement rendering logic in frontend/js/ui.js and charts.js that dynamically creates and injects DOM elements for the workflow timeline, approval banner, correction recommendations, and validation verdict.
- Wire the "Approve & Validate" and "Reject" buttons to re-submit the investigation request with the updated approval_status, and re-render the resulting state.

Provide Report Export:

- Implement a "Copy to Clipboard" and "Download JSON" action for the compiled final report, so users can export or share investigation results.



Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Autonomous Data Quality Investigation Agent locally to verify correct operation of the full multi-agent workflow, from dataset upload through investigation to validation.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Create and activate a virtual environment, then install all required dependencies from requirements.txt:

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root (based on .env.example) and add your Gemini API key and model name. The app loads this automatically using python-dotenv at startup.

```
# Get your key at: https://aistudio.google.com/app/apikey
LLM_API_KEY=*****

# Optional - defaults to gemini-1.5-flash if not set
LLM_MODEL=gemini-1.5-flash

# --- Application Settings ---
APP_ENV=development
APP_DEBUG=false
APP_HOST=0.0.0.0
APP_PORT=8000

# --- Database (required in future phases) ---
# DATABASE_URL=postgresql://user:password@localhost:5432/dqa_db

# --- Logging ---
LOG_LEVEL=INFO
```

Activity 5.2: Local Testing and Verification

Start the FastAPI Development Server:

- Run the application locally using:

```
uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

- Once running, open a browser and navigate to "http://127.0.0.1:8000/" to access the dashboard, or "http://127.0.0.1:8000/docs" for the interactive Swagger API documentation.

```
(.venv) C:\projects\SB projects\Autonomous Data Quality Investigation Agent\autonomous-data-quality-agent>uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\projects\\SB projects\\Autonomous Data Quality Investigation Agent\\autonomous-data-quality-agent']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [25420] using WatchFiles
INFO: Started server process [2208]
INFO: Waiting for application startup.
[startup] Autonomous Data Quality Investigation Agent - Phase 11 | env=development | LLM configured=False
INFO: Application startup complete.
```

- Upload a sample dataset from data/raw/ (such as sales_problematic.csv) and step through the workflow — profiling, anomaly detection, schema analysis, root cause investigation, correction recommendation, approval, and validation — to verify each agent's output is correctly generated and rendered.

- Alternatively, run the end-to-end CLI demo to exercise both the approved and pending-approval scenarios:

```
python examples/end_to_end_demo.py --scenario a
```

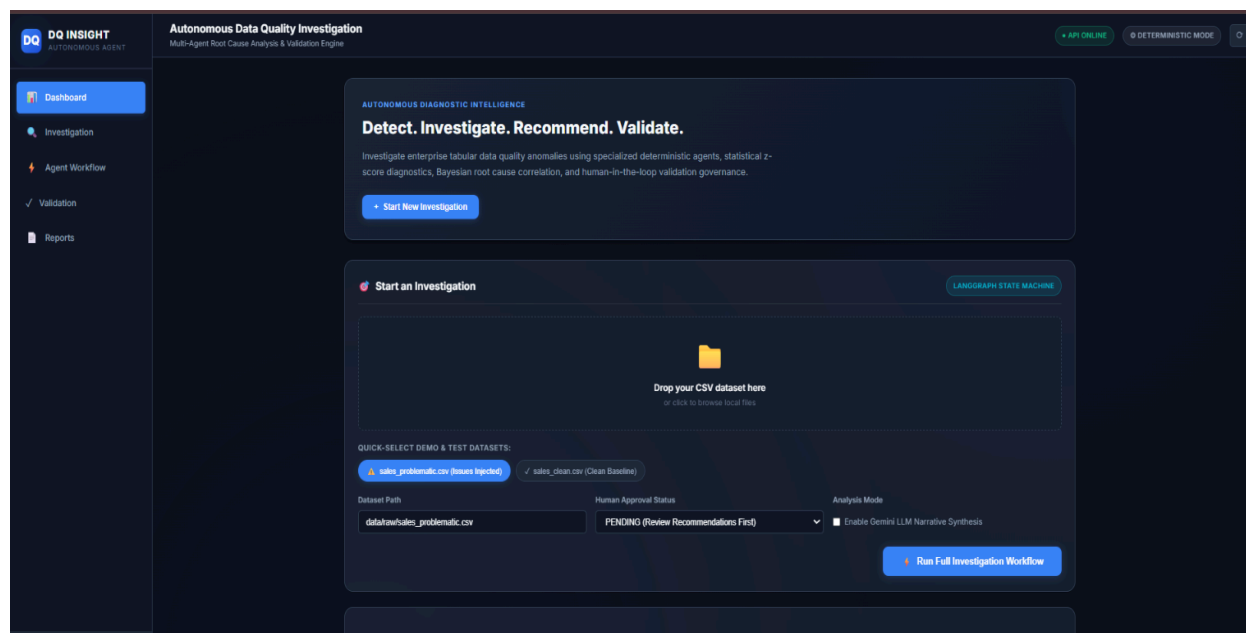
```
python examples/end_to_end_demo.py --scenario b
```

- Run the full automated test suite to confirm all phases pass:

```
pytest -v
```

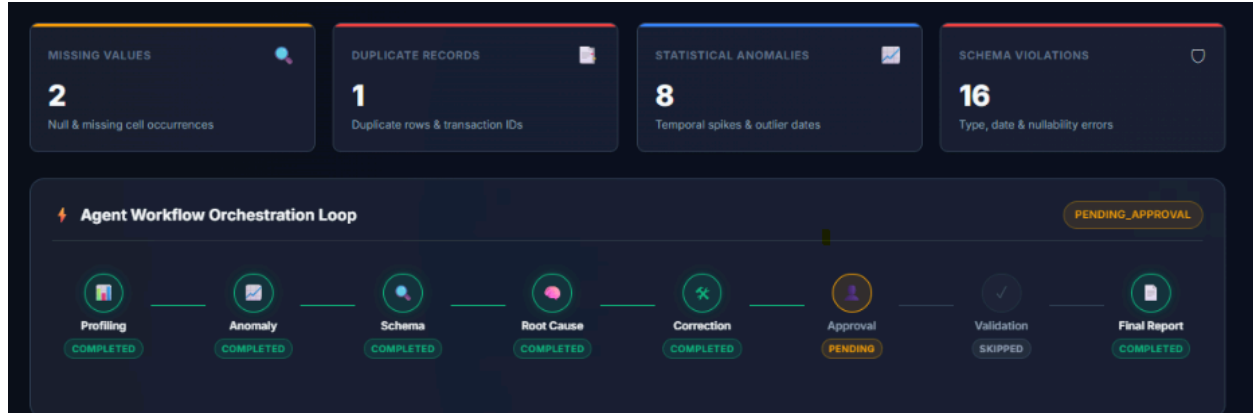
Exploring the Website's Web Pages

Dashboard Page (index.html):



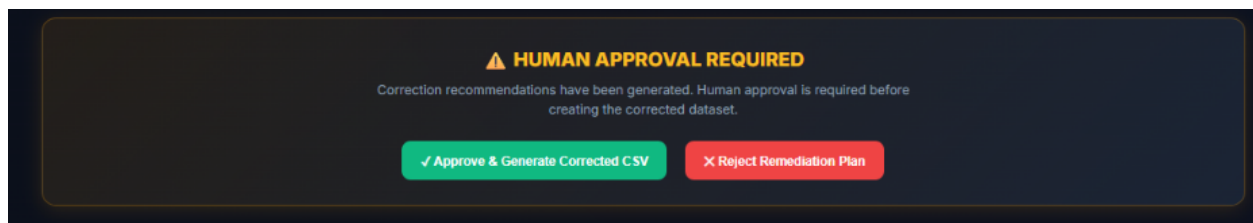
Description: The single-page interface of the Autonomous Data Quality Investigation Agent, titled "DQ Insight." It serves as both the landing page and the investigation tool. It features a dark, high-contrast dashboard design with a hero section, an investigation input card, and a placeholder empty state — all on one seamless page before an investigation is run.

Workflow Timeline Section:



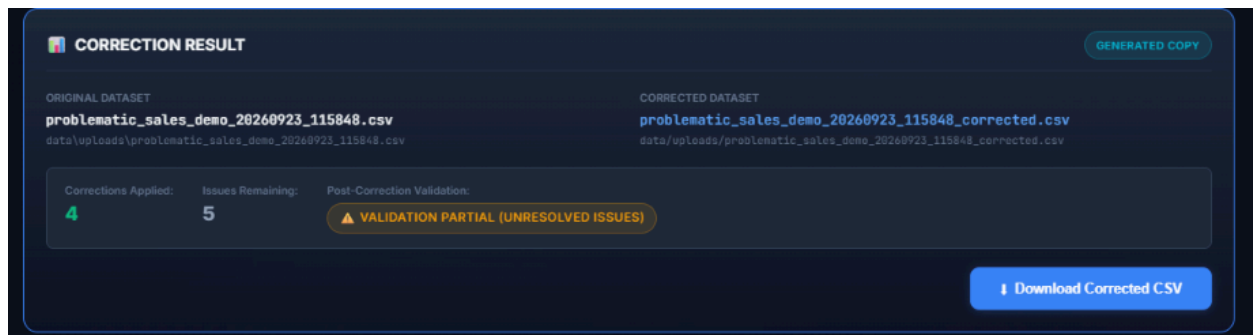
Description: Revealed once an investigation runs, this card displays a dynamic stepper visualizing each agent's execution state — Profiling, Anomaly Detection, Schema Analysis, Root Cause, and Correction — marked as COMPLETED, RUNNING, SKIPPED, or FAILED as the LangGraph workflow progresses.

Human Approval Gate Banner:



Description: An interactive banner presenting the Correction Recommendation Agent's prioritized action plan, offering one-click "Approve & Validate" or "Reject" controls so a human reviewer decides whether a correction proceeds.

Correction Result Section:



Description: Displayed once a correction is approved, this card highlights the recommended fixes and their risk ratings in an accent-bordered panel distinct from the rest of the dashboard.

Audit Section:

Remediation Audit & Data Changes

HUMAN-IN-THE-LOOP AUDIT

Human Approval Record

Approver

Human Operator

Status

APPROVED

Timestamp (UTC)

23/09/2026, 11:59:59

Approved Remediation Actions

No specific actions approved.

Data Change Summary

Category

UNRESOLVED

Changed Cells

0

Removed Duplicates

2

New Missing Values

0

Unresolved Values

5

Description: A card surfacing the full evidence trail behind the investigation — the deterministic findings from each agent that fed into the Root Cause Agent's hypotheses — for traceability and compliance review.

Validation Section:

✓ Post-Correction Validation Dashboard

DETERMINISTIC VERIFICATION

⚠ VALIDATION PARTIAL

2 Passed • 2 Failed • 1 Partial

PARTIAL

Independent Check Suite

✓ Duplicate Transaction Validation (duplicates)

0 duplicate transaction IDs/rows remaining (down from 2 before correction).

PASS

✗ Required Field Completeness Validation (completeness)

2 nullability violations remaining.

FAIL

✓ Schema Type Conformance Validation (schema)

All columns match expected logical data types without string suffixes (down from 1).

PASS

✗ Date Format & Timestamp Validation (schema)

1 date format issues remaining.

FAIL

⚠ Business Metric Revenue Validation (metrics)

Post-correction total revenue recalculated at \$10,103,725.29 (baseline was \$10,107,750.19).

PARTIAL

Before / After Metric Comparison

DUPLICATES

100% IMPROVEMENT

2 → 0

NULLABILITY VIOLATIONS

0% IMPROVEMENT

2 → 2

TYPE MISMATCHES

100% IMPROVEMENT

1 → 0

FORMAT ISSUES

0% IMPROVEMENT

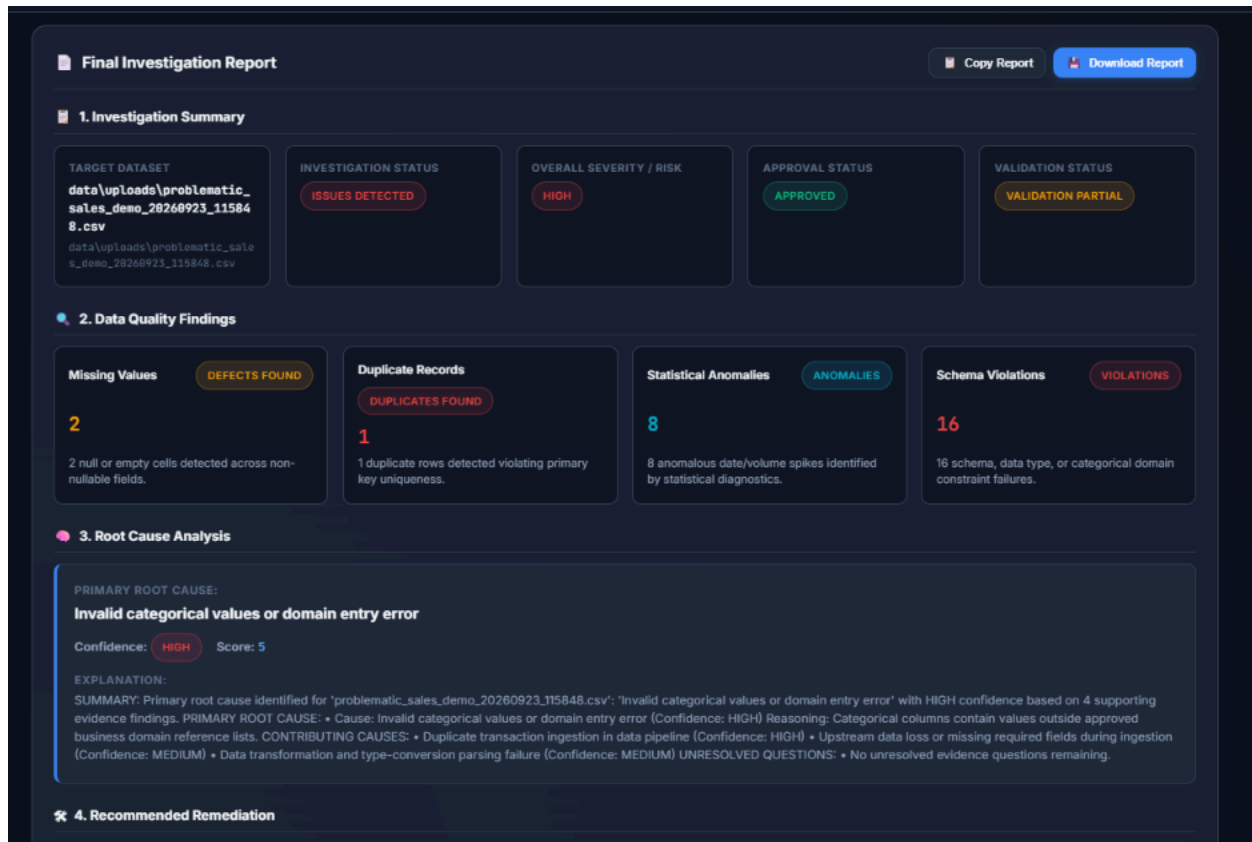
1 → 1

TOTAL REVENUE

10,187,750.19 → 10,103,725.29

Description: Shown after the Validation Agent runs, this card presents the PASS/PARTIAL/FAILED verdict alongside a before/after comparison table tracking duplicates, nullability, type mismatches, and revenue recalculations.

Reports Section:



The screenshot displays the 'Final Investigation Report' interface. At the top right, there are buttons for 'Copy Report' and 'Download Report'. The report is organized into four main sections:

- 1. Investigation Summary:** Contains five cards:
 - Target Dataset:** data\uploads\problematic_sales_demo_20260923_115848.csv
 - Investigation Status:** ISSUES DETECTED
 - Overall Severity / Risk:** HIGH
 - Approval Status:** APPROVED
 - Validation Status:** VALIDATION PARTIAL
- 2. Data Quality Findings:** Contains four cards:
 - Missing Values:** 2 DEFECTS FOUND. 2 null or empty cells detected across non-nullable fields.
 - Duplicate Records:** 1 DUPLICATES FOUND. 1 duplicate rows detected violating primary key uniqueness.
 - Statistical Anomalies:** 8 ANOMALIES. 8 anomalous date/volume spikes identified by statistical diagnostics.
 - Schema Violations:** 16 VIOLATIONS. 16 schema, data type, or categorical domain constraint failures.
- 3. Root Cause Analysis:**
 - Primary Root Cause:** Invalid categorical values or domain entry error
 - Confidence:** HIGH **Score:** 5
 - Explanation:** SUMMARY: Primary root cause identified for "problematic_sales_demo_20260923_115848.csv": "Invalid categorical values or domain entry error" with HIGH confidence based on 4 supporting evidence findings. PRIMARY ROOT CAUSE: • Cause: Invalid categorical values or domain entry error (Confidence: HIGH) Reasoning: Categorical columns contain values outside approved business domain reference lists. CONTRIBUTING CAUSES: • Duplicate transaction ingestion in data pipeline (Confidence: HIGH) • Upstream data loss or missing required fields during ingestion (Confidence: MEDIUM) • Data transformation and type-conversion parsing failure (Confidence: MEDIUM) UNRESOLVED QUESTIONS: • No unresolved evidence questions remaining.
- 4. Recommended Remediation:** (Section header visible)

Description: The final card on the dashboard, offering "Copy to Clipboard" and "Download JSON" actions so users can export or share the compiled Final Workflow Report.

Conclusion:

The Autonomous Data Quality Investigation Agent is an agentic AI platform built with FastAPI, LangGraph, and a dark-themed HTML/CSS/JavaScript dashboard that automates the diagnosis, investigation, and validation of data quality defects in tabular datasets. It combines deterministic Pandas/NumPy profiling, anomaly detection, and schema analysis tools with Google's Gemini API for plain-language interpretation, a Human Approval Gate to keep corrections under human control, and an independent Validation Agent to verify results — all orchestrated through a LangGraph state machine. The project, which included model selection, multi-agent development, backend orchestration, frontend dashboard development, and deployment, highlights how targeted AI, deterministic guardrails, and a structured multi-agent framework can turn a simple CSV upload into a fully audited data quality investigation, with 286 automated tests passing. Looking ahead, the platform offers significant opportunities for expansion, such as supporting additional dataset schemas beyond sales data, persisting investigation history to a database, and adding scheduled or streaming data quality monitoring. By continuously refining the underlying agent prompts and expanding the dashboard's visualizations, the platform is well-positioned to evolve from a per-dataset investigation tool into a continuous data quality assurance assistant for data teams of all sizes.